

八股问题

1 机器学习中的损失函数

机器学习中的损失函数根据任务类型可以分为以下几类:

1.1 回归任务损失函数

- 均方误差(MSE): $L = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$, 对异常值敏感
- 平均绝对误差(MAE): $L = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$, 对异常值更鲁棒
- Huber Loss: 结合MSE和MAE的优点, 在误差较小时使用MSE, 误差较大时使用MAE

1.2 分类任务损失函数

- 交叉熵损失(Cross Entropy):
 - 二分类: $L = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$
 - 多分类: $L = -\frac{1}{n} \sum_{i=1}^n \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c})$
- Focal Loss: $L = -\sum_{i=1}^n \alpha_i (1 - \hat{y}_i)^\gamma y_i \log(\hat{y}_i)$, 解决类别不平衡
- KL散度: $D_{KL}(P||Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$, 衡量两个分布的差异

1.3 对比学习损失函数

- InfoNCE Loss: $L = -\log \frac{\exp(\text{sim}(z_i, z_i^+)/\tau)}{\sum_{j=1}^N \exp(\text{sim}(z_i, z_j)/\tau)}$

2 交叉熵作为损失函数的原理及相关概念

2.1 为什么交叉熵能作为损失函数

交叉熵能作为损失函数的核心原因有以下几点:

1. 最大似然估计的等价性

最小化交叉熵等价于最大化似然函数。假设样本独立同分布, 似然函数为:

$$L(\theta) = \prod_{i=1}^n P(y_i | x_i; \theta)$$

取负对数得到:

$$-\log L(\theta) = -\sum_{i=1}^n \log P(y_i|x_i; \theta)$$

这正是交叉熵损失的形式。所以最小化交叉熵就是最大化似然,有坚实的统计学基础。

2. 梯度性质良好

交叉熵配合softmax激活函数,梯度形式简洁:

$$\frac{\partial L}{\partial z_i} = \hat{y}_i - y_i$$

梯度大小正好等于预测误差,不会出现梯度消失问题,训练稳定高效。

3. 凸优化性质

对于线性模型,交叉熵是凸函数,保证能找到全局最优解。

2.2 InfoNCE与交叉熵的关系

InfoNCE(Noise Contrastive Estimation)本质上是多分类交叉熵的一种特殊形式。

InfoNCE的形式:

$$L_{InfoNCE} = -\log \frac{\exp(\text{sim}(z_i, z_i^+)/\tau)}{\sum_{j=1}^N \exp(\text{sim}(z_i, z_j)/\tau)}$$

可以看出,这就是一个N分类的交叉熵:

- 正样本 z_i^+ 对应真实类别,标签为1
- 其他N-1个负样本对应其他类别,标签为0
- 分母是softmax归一化

所以InfoNCE = 把对比学习转化为N分类问题,用交叉熵优化。核心思想是最大化正样本对的相似度,同时最小化负样本对的相似度。

2.3 KL散度与交叉熵的关系

KL散度和交叉熵有紧密的数学关系:

$$D_{KL}(P||Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)} = \sum_i P(i) \log P(i) - \sum_i P(i) \log Q(i)$$

$$D_{KL}(P||Q) = -H(P) + H(P, Q)$$

其中 $H(P)$ 是P的熵, $H(P, Q)$ 是交叉熵。

关键结论:

- KL散度 = 交叉熵 - 熵
- 在分类任务中,真实标签P是固定的one-hot分布,熵 $H(P) = 0$
- 因此最小化交叉熵等价于最小化KL散度
- KL散度非负,当且仅当 $P=Q$ 时为0

所以在分类任务中,最小化交叉熵就是让模型预测分布 Q 尽可能接近真实分布 P ,这正是我们想要的。
区别:

- KL散度不对称: $D_{KL}(P||Q) \neq D_{KL}(Q||P)$
- 交叉熵也不对称,但在分类任务中 P 固定,只优化 Q
- KL散度更多用于分布匹配(如VAE),交叉熵更多用于分类任务

3 PPO算法详解及其在大模型后训练中的应用

3.1 PPO算法原理

PPO(Proximal Policy Optimization)是目前最流行的强化学习算法,由OpenAI在2017年提出。

核心思想: 在策略更新时,限制新策略和旧策略的差异不要太大,避免训练不稳定。

目标函数:

$$L^{CLIP}(\theta) = \mathbb{E}_t[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)]$$

其中:

- $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ 是重要性采样比率
- $\hat{A}_t$ 是优势函数估计
- ϵ 是裁剪参数,通常取0.1或0.2

关键机制:

1. **裁剪(Clipping):** 当 r_t 超出 $[1 - \epsilon, 1 + \epsilon]$ 范围时,梯度被截断,防止策略更新过大
2. **优势函数:** 用GAE(Generalized Advantage Estimation)估计,减小方差
3. **多轮更新:** 同一批数据可以重复使用多次,提高样本效率

算法流程:

1. 用当前策略 $\pi_{\theta_{old}}$ 收集轨迹数据
2. 计算每个时间步的优势函数 $\hat{A}_t$
3. 用收集的数据更新策略,最大化 L^{CLIP}
4. 重复K轮(通常K=3-10)
5. 更新 $\theta_{old} \leftarrow \theta$,进入下一轮采样

3.2 为什么大模型后训练选择PPO

大模型的RLHF(Reinforcement Learning from Human Feedback)后训练几乎都选择PPO,原因如下:

- 1 训练稳定性: 大模型参数量巨大(几十亿到千亿),训练极其不稳定, PPO的**裁剪机制**保证策略更新幅度可控,避免灾难性遗忘。
- 2 工程实现友好: 相比TRPO,PPO实现简单,不需要计算二阶导数和共轭梯度; 支持**批量并行采样**,充分利用GPU集群
- 3 样本效率: 大模型生成样本成本极高(需要推理生成文本), PPO是**on-policy**算法,但允许多轮更新同一批数据,实现了稳定性与效率的平衡
- 4 超参数鲁棒性: 大模型训练成本高,不能频繁调参,而PPO对超参数不敏感

3.3 RLHF中的PPO应用

在ChatGPT等大模型的RLHF训练中,PPO的具体应用流程:

- 1 阶段1: 监督微调(SFT)———用高质量人工标注数据微调预训练模型, 得到初始策略 π_{SFT}
- 2 阶段2: 奖励模型训练(RM)———人工对模型生成的多个回复进行排序, 训练奖励模型 $r_\phi(x, y)$ 预测人类偏好
- 3 阶段3: PPO强化学习———用户输入prompt x , 模型生成回复 $y \sim \pi_\theta(y|x)$, 计算奖励 $r(x, y) = r_\phi(x, y) - \beta \log \frac{\pi_\theta(y|x)}{\pi_{SFT}(y|x)}$ (第二项是KL惩罚,防止模型偏离SFT太远), 用PPO最大化期望奖励

总结: PPO在**稳定性、样本效率、实现简单性**之间达到了最佳平衡,是大模型RLHF的不二之选。

4 MCP和Skill的定义与联系

MCP和Skill是两种不同的能力扩展机制, 一句话总结:「MCP 解决”连接”问题: 让 AI 能访问外部世界」「Skill 解决”方法论”问题: 教 AI 怎么做某类任务」

- 1 MCP定义: MCP是一种标准化协议,允许AI系统通过统一接口调用外部工具和服务 (官方的比喻是” AI 应用的 type-C 接口”——就像 type-C 提供了一种通用的方式连接各种设备, MCP 提供了一种通用的方式连接各种工具和数据源。)
- 2 为什么需要MCP: 大模型本身**无法直接操作外部系统**(数据库、文件系统、API),MCP提供了**行动能力**; 其次大模型的知识截止于训练时间,MCP可以**获取最新数据**; 大模型在复杂计算上容易出错,MCP可以调用**专业工具**(计算器、代码执行器); 无需修改AI核心,通过**添加MCP服务器**即可扩展新能力
- 3 Skill定义: Skill是预定义的**指令集或提示模板**,增强AI在特定领域的能力,直接注入到**AI的上下文**中,成为系统提示的一部分,所谓 “一次实现, 到处使用”
- 4 为什么需要Skill: 首先, Skill可以注入领域专家的经验 and 最佳实践使得大模型在指定领域表现卓越; 其次, Skill可以**引导AI的推理方向**,通过明确的指令和约束, 避免偏离任务目标或产生不相关的输出; 并且, 大模型的训练数据有时效性,Skill可以**补充最新知识**

5 两者通常配合使用

Skill + MCP的协同价值

- 知识与行动结合: Skill提供”知道怎么做”,MCP提供”能够做”
- 完整的工作流: Skill定义流程,MCP执行具体操作,形成闭环
- 质量保证: Skill确保决策质量,MCP确保执行准确性

总结: Skill和MCP分别解决了AI的”智力”和”能力”问题。Skill让AI智力更高(知道该做什么、怎么做),MCP让AI能力更强(能够操作真实世界)。两者结合,才能构建真正实用的AI系统。

5 传统强化学习Agent与大模型Agent的区别

5.1 传统强化学习Agent

核心特征:

- 学习方式: 通过试错与环境交互,从奖励信号中学习策略
- 状态表示: 通常是低维向量或离散状态(如游戏画面、机器人传感器数据)
- 动作空间: 明确定义的有限动作集(如上下左右、关节角度)
- 策略形式: 神经网络映射状态到动作概率分布
- 优化目标: 最大化累积奖励
- 泛化能力: 局限于训练环境,难以迁移到新任务

典型应用: AlphaGo(围棋)、Atari游戏、机器人控制

5.2 大模型Agent

核心特征:

- 学习方式: 基于预训练+微调,从海量文本中学习世界知识和推理能力
- 状态表示: 自然语言描述的复杂场景,可以理解抽象概念
- 动作空间: 开放式动作,可以生成任意文本、调用工具、执行代码
- 策略形式: 语言模型生成下一步行动的自然语言描述
- 优化目标: 完成用户指定的任务目标,可以是多步推理、规划、执行
- 泛化能力: 强大的零样本/少样本学习能力,可以处理未见过的任务

典型应用: ChatGPT、AutoGPT、代码助手、智能客服

5.3 二者不同

- (1) 训练方式不同：传统agent能力来自于交互中的试错；而大模型agent依赖于预训练得到的语言知识 + 提示词 + 工具调用 + 外部记忆
- (2) 环境类型不同：传统agent适用于状态清晰、动作可枚举、奖励可定义的问题；而大模型agent适用于开放世界、非结构化、语言主导的任务
- (3) 动作空间不同：传统agent动作通常是离散或连续控制信号，而大模型agent的动作更像高层语义动作
- (4) 总结：传统强化学习智能体是“以奖励驱动的策略学习器”，大模型智能体是“以大模型为中枢的任务执行系统”；前者重点在学策略，后者重点在组织知识、规划步骤、调用工具和完成复杂开放任务。

6 AI Coding的Plan模式与Agent模式区别

在AI Coding系统中，Plan模式与Agent模式代表了两种不同的任务组织方式。前者强调先形成较完整的实现方案，再按既定思路执行；后者则强调在执行过程中持续获取反馈，并根据中间结果动态调整后续动作。

6.1 Plan模式

Plan模式的核心是“先规划，后执行”。当用户提出任务后，系统先分析需求，明确需要修改的文件、涉及的函数以及大致实施步骤，在此基础上再生成或修改代码。其优势在于流程清晰、成本较低、执行效率较高，因而更适合目标明确、影响范围有限的任务，例如局部代码生成、简单重构、变量重命名以及格式化修复等。

不过，Plan模式对前期规划质量较为依赖。如果任务本身存在较强不确定性，或者执行过程中需要频繁依赖编译结果、测试反馈和运行日志来修正判断，那么预先给出的方案可能很快失效，因此其应对复杂场景的能力相对有限。

6.2 Agent模式

Agent模式的核心是“边执行，边调整”。系统通常先观察当前代码库状态、错误信息或测试结果，再决定下一步行动，例如读取文件、搜索符号、修改代码、运行测试或调用外部工具；随后根据新的反馈继续判断和修正，直至任务完成。其本质是一种“观察—行动—反馈—再决策”的闭环过程。

相较于Plan模式，Agent模式更适合复杂缺陷修复、多文件协同修改、需要调试验证的功能实现以及与外部系统交互的任务。其优势在于能够处理不确定性并具备较强自适应能力，但代价是推理轮次更多、工具调用更频繁，因此整体成本和时延通常也更高。

6.3 两种模式的区别与实际应用

总体而言，Plan模式更强调执行前的集中规划，适合简单、明确、可控的任务；Agent模式更强调执行中的动态决策，适合复杂、开放、依赖反馈的任务。前者通常更快、更省成本，但对异常情况较为敏感；后者通常更稳健、更灵活，但系统复杂度和资源开销也更高。

在实际AI Coding系统中，二者往往并非完全割裂，而是以混合方式共同存在。对于多数中高复杂度的AI Coding任务，更有效的使用方式通常不是单独采用Plan模式或Agent模式，而是先由plan生成可审阅的任务计划，以明确整体修改方向、影响范围与验证路径，再由Agent在执行过程中结合编译结果、测试反馈与环境状态进行动态调整。对于简单且边界清晰的小规模任务，则可直接进入Agent执行阶段，以避免过度规划带来的额外开销。